

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 — Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 — Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	M. SANA FATHIMA	Reg. No.:	192371082
Date:	11/08/2026	Duration:	60 minutes

Code of Conduct I **M. SANA FATHIMA(192371082)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: M. SANA FATHIMA

Annexure A — Git & Docker Technical Assignment

Assigned Problem Statement 9: Smart E-commerce Order Fulfillment Platform

1. Problem Analysis

Problem Statement

Online retailers that rely on manual coordination between their storefront, warehouse staff, and shipping partners suffer from delayed order processing, overselling of out-of-stock items, and poor visibility into where an order is in its lifecycle. The assigned project, the Smart E-commerce Order Fulfillment Platform, is a full-stack web application that automates the order lifecycle end-to-end: cart checkout, real-time inventory reservation, warehouse pick-list generation, shipment tracking, and automated customer notifications. The platform is built with a React frontend, a Node.js/Express order-api backend, a dedicated notification-service worker, a PostgreSQL database, and a Redis queue/cache, and is containerized with Docker so it can be developed, tested, and deployed consistently across environments.

Project Objectives

- Automate the order lifecycle from checkout to delivery, replacing manual, spreadsheet-driven coordination.
- Prevent overselling by reserving inventory in real time as soon as an order is placed.
- Assign orders to the correct warehouse and generate accurate pick lists for fulfillment staff.
- Keep customers informed with automated status notifications at each stage of fulfillment.
- Use Git for collaborative, versioned development and Docker for consistent, portable deployment.

Functional Requirements

- FR1 — Customers can browse products, add items to a cart, and complete checkout.
- FR2 — On order placement, the system reserves stock for each item and rejects the order if insufficient stock exists.
- FR3 — The order-api assigns each order to a warehouse and generates a pick list for fulfillment staff.
- FR4 — Order status transitions (placed → packed → shipped → delivered) are tracked and timestamped.
- FR5 — The notification-service sends an email/SMS to the customer whenever order status changes.
- FR6 — Administrators can view a dashboard of live orders, inventory levels, and fulfillment performance.

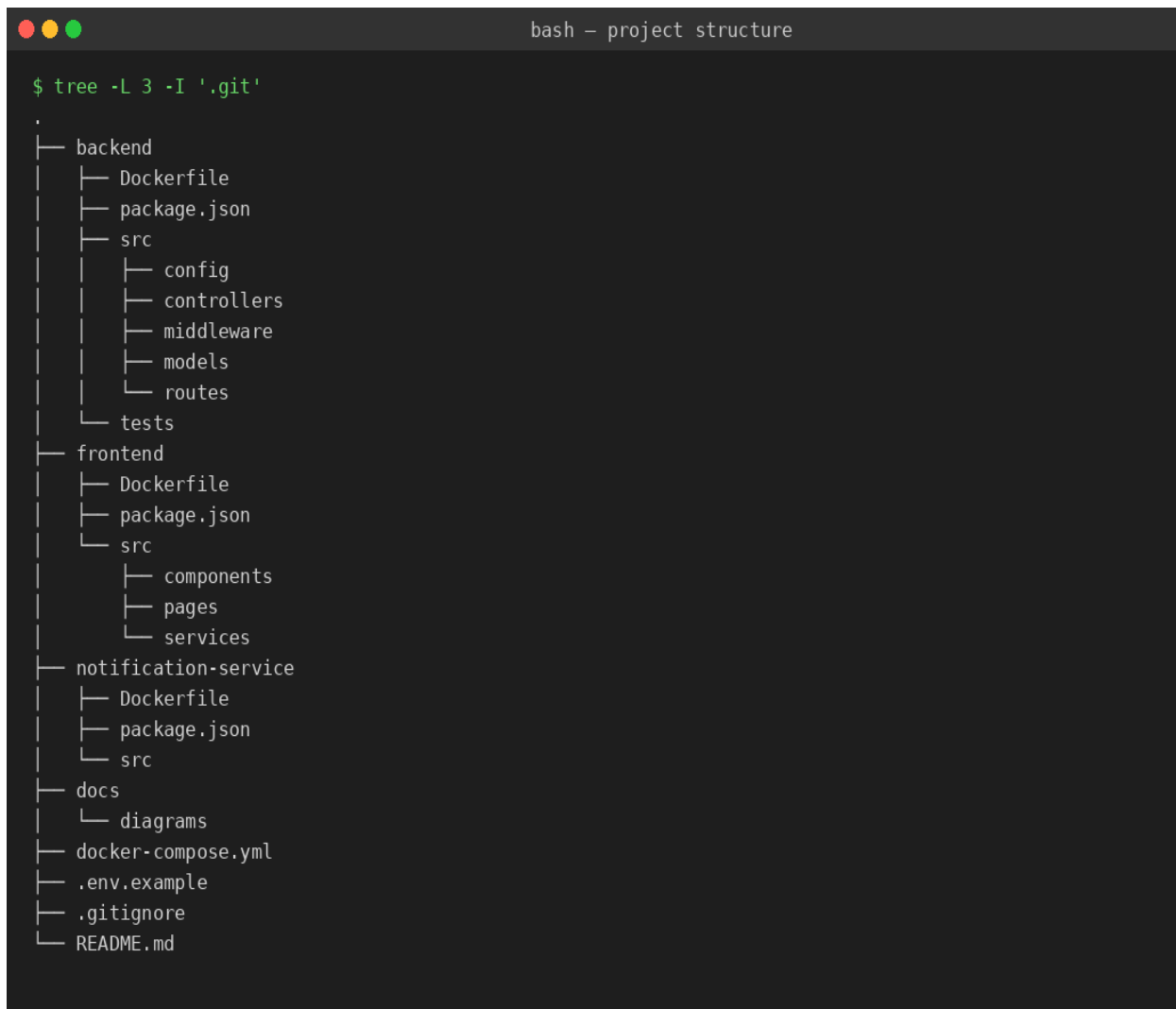
Expected Outcomes

- A fully containerized multi-service application (frontend, order-api, notification-service, database, cache) deployable with a single command.
- A Git repository with a clean Gitflow branching history demonstrating collaborative development practices.
- Reduced order-processing delays and eliminated overselling compared to manual, spreadsheet-based coordination.

- A reproducible development-to-production pipeline enabled by Docker and Docker Compose.
- To main and create a develop branch as the integration branch

2. Project Structure Design

The repository follows a monorepo structure that separates the frontend, backend (order-api), and notification-service into independently buildable units, each with its own Dockerfile, while sharing a single Git history and a top-level docker-compose.yml that orchestrates them together.



```
bash - project structure

$ tree -L 3 -I '.git'
.
├── backend
│   ├── Dockerfile
│   ├── package.json
│   └── src
│       ├── config
│       ├── controllers
│       ├── middleware
│       ├── models
│       └── routes
├── frontend
│   ├── Dockerfile
│   ├── package.json
│   └── src
│       ├── components
│       ├── pages
│       └── services
├── notification-service
│   ├── Dockerfile
│   ├── package.json
│   └── src
├── docs
│   └── diagrams
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

Figure 1 — Repository folder structure for the Smart E-commerce Order Fulfillment Platform

Purpose of Major Folders and Files

Folder / File	Purpose
backend/src/controllers/	Request handlers containing business logic for orders, inventory reservation, and checkout.
backend/src/models/	Schemas defining Order, Product, and Warehouse data structures.
backend/src/routes/	Express route definitions mapping HTTP endpoints (e.g., /api/orders) to controllers.
backend/src/middleware/	Reusable middleware for authentication, error handling, and payment-gateway failure handling.
backend/src/config/	Database connection and environment configuration.
backend/tests/	Unit and integration tests for order and inventory logic.
backend/Dockerfile	Instructions to build the order-api container image.
frontend/src/components/	Reusable UI components (e.g., CartSummary, OrderTracker).
frontend/src/pages/	Top-level pages such as Dashboard and Checkout.
frontend/src/services/	Axios-based API client modules that call the order-api REST API.

3. Git Repository Initialization

- Repository Initialization Steps
- Create the project root folder and run git init to create the local .git repository.
- Rename the default branch to main and create a develop branch as the integration branch.
- Add the initial project scaffold (backend/, frontend/, notification-service/, .gitignore, README.md) and make the first commit.

```
bash - git init

$ mkdir order-fulfillment-platform && cd order-fulfillment-platform
$ git init
Initialized empty Git repository in .git/
$ git branch -m main
$ mkdir -p backend frontend notification-service docs/diagrams
$ echo "node_modules/\ndist/\nbuild/\n.env\n*.log" > .gitignore
$ git add .gitignore README.md backend frontend notification-service docker-compose.yml .env.example docs
$ git commit -m "chore: initial project scaffold for order fulfillment platform"
[main a64e4bb] chore: initial project scaffold for order fulfillment platform
21 files changed, 8 insertions(+)
$ git checkout -b develop
Switched to a new branch 'develop'
$ git remote add origin https://github.com/team/order-fulfillment-platform.git
$ git push -u origin main
$ git push -u origin develop
```

Figure 2 — Terminal commands used to initialize the Git repository

Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by git init; stores the entire object database, refs, and history.
.gitignore	Lists files/folders (node_modules/, .env, dist/, build/) that Git should never track.
README.md	Documents project purpose, setup steps, and usage for collaborators.
.env.example	Template of required environment variables, committed safely without real secret values.
package.json / package-lock.json	Tracks Node.js dependencies and locks exact versions for reproducible installs.

4. Git Branching Strategy

Selected Branching Model: Gitflow

The project uses a Gitflow-based branching strategy with five branch types: main, develop, feature/*, release/*, and hotfix/*.

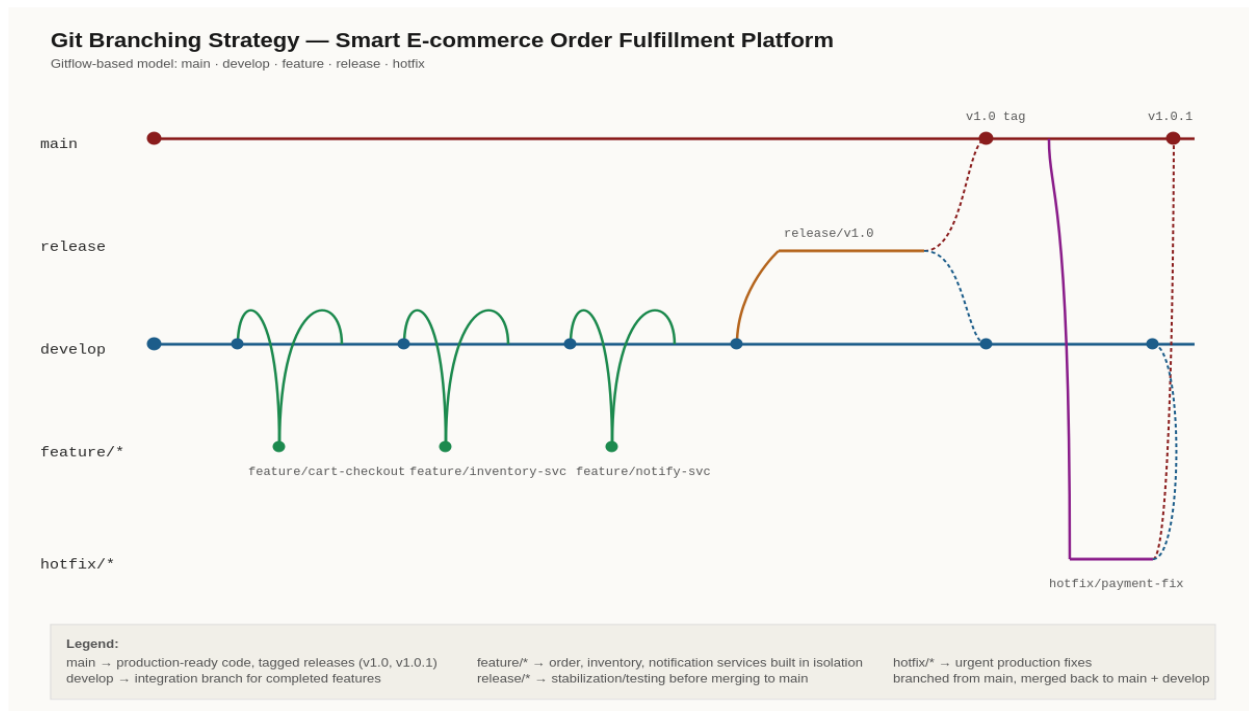


Figure 3 — Git branching strategy (Gitflow) for the order fulfillment platform

Why This Model Fits the Project

- Independent modules (checkout, inventory reservation, notifications, dashboard) map naturally to isolated feature branches, reducing merge conflicts between team members.
- The develop branch provides a stable integration point for continuous testing before anything reaches production.
- The release branch allows QA and documentation to happen without blocking new feature work on develop.

5. Version Control Workflow

The example below shows a complete feature cycle for the inventory-reservation route: branching from develop, committing incremental changes on two parallel feature branches, and merging back with a real conflict resolved along the way.

```
bash - feature branch workflow

$ git checkout -b feature/inventory-svc develop
Switched to a new branch 'feature/inventory-svc'
$ git add backend/src/routes/orderRoutes.js
$ git commit -m "feat(inventory): add stock reservation route"
[feature/inventory-svc 9c1d481] feat(inventory): add stock reservation route
$ git push origin feature/inventory-svc

$ git checkout develop
$ git add backend/src/routes/orderRoutes.js
$ git commit -m "feat(orders): include item count in order list response"
[develop de46743] feat(orders): include item count in order list response

$ git merge --no-ff feature/inventory-svc
Auto-merging backend/src/routes/orderRoutes.js
CONFLICT (content): Merge conflict in backend/src/routes/orderRoutes.js
Automatic merge failed; fix conflicts and then commit the result.

$ cat backend/src/routes/orderRoutes.js
<<<<<< HEAD
router.get('/', (req, res) => res.json({ status: 'ok', items: [], count: 0 }));
=====
router.get('/', (req, res) => res.json({ status: 'ok', items: [] }));
router.post('/reserve', (req, res) => res.json({ reserved: true }));
>>>>>> feature/inventory-svc

# Manually reconciled both changes, kept count and reserve route
$ git add backend/src/routes/orderRoutes.js
$ git commit -m "merge: resolve route conflict, merge feature/inventory-svc into develop"
[develop b320e53] merge: resolve route conflict, merge feature/inventory-svc into develop
$ git push origin develop
```

Figure 4 — Feature branch workflow including conflict resolution

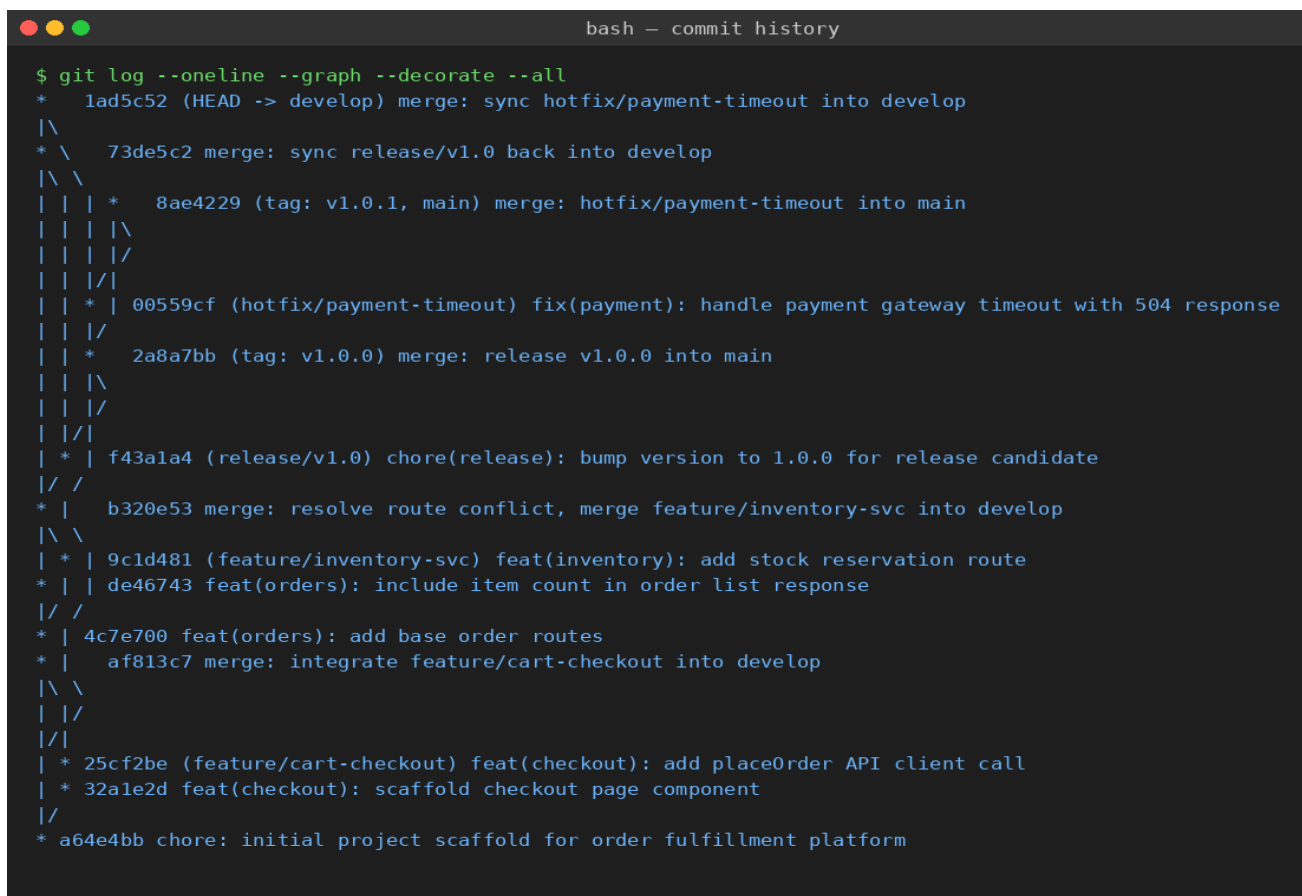
Purpose of Each Git Operation

Operation	Purpose
git checkout -b feature/x	Creates and switches to a new isolated branch for a single feature.
git add	Stages specific changed files for the next commit.
git commit -m "..."	Records a snapshot of staged changes with a descriptive message.
git push origin <branch>	Uploads local commits to the remote repository (GitHub).

Operation	Purpose
Pull Request	Requests review and discussion before merging into develop, enabling collaboration.
git merge --no-ff	Merges the feature branch into develop, preserving a merge commit for traceability.

6. Commit History Analysis

Project Commit History



```

bash – commit history

$ git log --oneline --graph --decorate --all
* 1ad5c52 (HEAD -> develop) merge: sync hotfix/payment-timeout into develop
|\
* \ 73de5c2 merge: sync release/v1.0 back into develop
|\ \
| | | * 8ae4229 (tag: v1.0.1, main) merge: hotfix/payment-timeout into main
| | | |\
| | | | /
| | | | /
| | | | /
| | * | 00559cf (hotfix/payment-timeout) fix(payment): handle payment gateway timeout with 504 response
| | | /
| | * 2a8a7bb (tag: v1.0.0) merge: release v1.0.0 into main
| | |\
| | | /
| | | /
| | | /
| * | f43a1a4 (release/v1.0) chore(release): bump version to 1.0.0 for release candidate
| / /
* | b320e53 merge: resolve route conflict, merge feature/inventory-svc into develop
|\ \
| * | 9c1d481 (feature/inventory-svc) feat(inventory): add stock reservation route
* | | de46743 feat(orders): include item count in order list response
| / /
* | 4c7e700 feat(orders): add base order routes
* | af813c7 merge: integrate feature/cart-checkout into develop
|\ \
| | /
| | /
| /
| * 25cf2be (feature/cart-checkout) feat(checkout): add placeOrder API client call
| * 32a1e2d feat(checkout): scaffold checkout page component
| /
* a64e4bb chore: initial project scaffold for order fulfillment platform

```

Figure 5 — Commit history graph

Commit Message Conventions

All commits follow the Conventional Commits format: `<type>(<scope>): <short description>`, keeping the subject line under ~72 characters and written in the imperative mood.

How This Contributes to Project Maintenance

- Each commit represents one logical, reviewable change, making git log and git blame useful for tracing why a line changed.
- Consistent prefixes (feat, fix, chore, test) allow automatic changelog generation and quick filtering of history.
- Small, frequent commits make git revert and git bisect effective for isolating the source of a regression.
- Descriptive messages reduce onboarding time for new team members reviewing project history.

7. Docker Environment Design

Why Docker Is Suitable for This Project

- The platform has multiple distinct runtimes (React/Node build tools, Node.js order-api, notification worker, PostgreSQL, Redis) — Docker isolates each so their dependencies never conflict.
- Fulfillment operations may run across different physical or cloud servers; Docker guarantees the application behaves identically regardless of host OS.
- docker compose up allows the entire multi-service stack to be started with a single command, drastically simplifying onboarding and demos.
- Images can be tagged and pushed to a registry, giving traceable, rollback-capable deployments tied to Git release tags.
- Containers can be scaled independently — e.g., running multiple order-api replicas behind a load balancer during peak sales events.
- The develop branch provides a stable integration point for continuous testing before anything reaches production.
- The release branch allows QA and documentation to happen without blocking new feature work on develop

Application Components Requiring Containerization

Component	Container / Reason
React frontend	Container: fulfillment-frontend. Built into static assets and served via NGINX; isolated from backend build tools.
Node.js/Express order-api	Container: fulfillment-order-api. Runs the REST API; needs its own Node runtime and npm dependencies.
notification-service	Container: fulfillment-notify-svc. Consumes queued jobs from Redis and sends order-status emails/SMS.
PostgreSQL	Container: fulfillment-database. Persists order, product, and warehouse data; isolated with its own data volume.

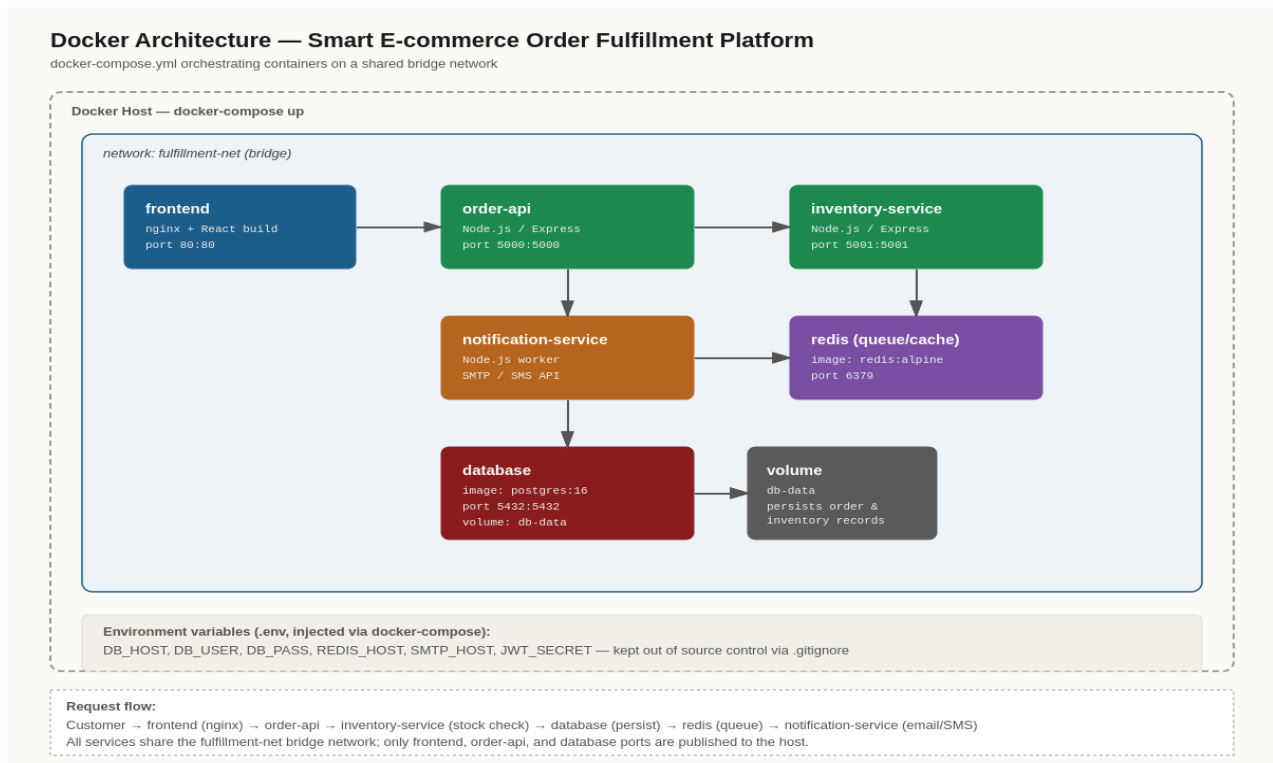


Figure 6 — Containerized system architecture for the Smart E-commerce Order Fulfillment Platform

8. Dockerfile Development

Backend Dockerfile (backend/Dockerfile)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 5000
CMD ["node", "src/server.js"]
```

Purpose of Each Instruction

Instruction	Purpose
FROM	Sets the base image (Node.js Alpine for build/runtime, NGINX Alpine to serve static files).
WORKDIR	Sets the working directory inside the container for subsequent instructions.
COPY package*.json ./	Copies only dependency manifests first so Docker can cache the npm install layer.
RUN npm ci	Installs exact dependency versions from package-lock.json for reproducible builds.
COPY . .	Copies the rest of the application source code into the image.
RUN npm run build	Compiles the React application into optimized static assets (frontend only).

9. Docker Compose Design

docker-compose.yml

```
version: "3.9"

services:
  frontend:
    build: ./frontend

    ports:
      - "80:80"

    depends_on:
      - order-api

    networks:
      - fulfillment-net

  order-api:
    build: ./backend

    ports:
      - "5000:5000"

    env_file:
      - ./backend/.env

    depends_on:
      - database
      - redis

    networks:
      - fulfillment-net

  notification-service:
    build: ./notification-service

    env_file:
      - ./notification-service/.env
```

Element	Explanation
Services	frontend (React + NGINX), order-api (Node/Express API), notification-service (worker), database (PostgreSQL), redis (queue/cache) — each built and run as an independent container.
Networking	All services join a shared bridge network fulfillment-net, so they can reach each other by service name (e.g., order-api connects to database:5432).
Volumes	db-data is a named volume mounted at /var/lib/postgresql/data so order and inventory records survive container restarts and rebuilds.
Environment Variables	order-api and notification-service load DB_HOST, DB_USER, DB_PASS, REDIS_HOST, SMTP_HOST, and JWT_SECRET from .env files via env_file, keeping secrets out of the image and out of Git.
Dependencies (depends_on)	order-api waits for database and redis, frontend waits for order-api, and notification-service waits for redis — so services start in the correct order.

10. Container Deployment and Testing

```

bash — docker compose up

$ docker compose build
[+] Building order-api          ... done
[+] Building inventory-service ... done
[+] Building notification-service ... done
[+] Building frontend          ... done

$ docker compose up -d
[+] Running 6/6
 - Network fulfillment-net          Created
 - Volume "order-fulfillment_db-data" Created
 - Container fulfillment-database   Started
 - Container fulfillment-redis      Started
 - Container fulfillment-order-api   Started
 - Container fulfillment-frontend   Started

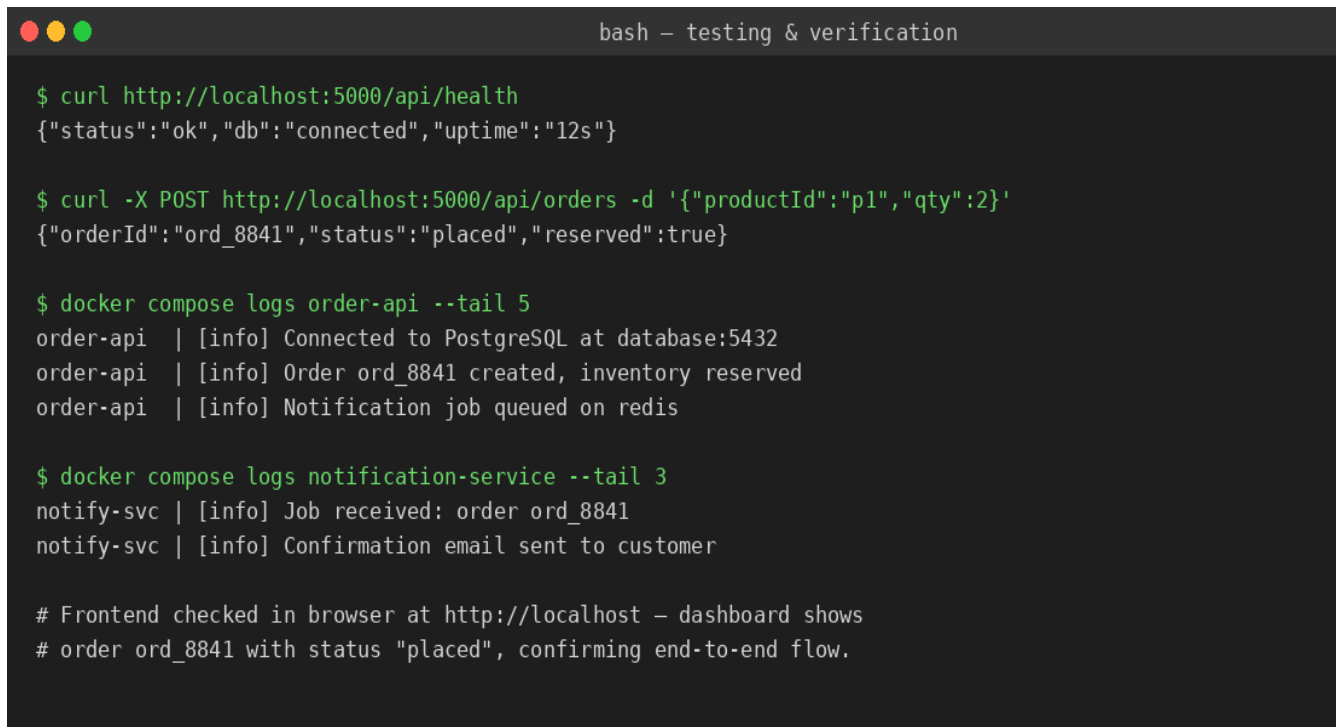
$ docker compose ps
NAME                                IMAGE                                STATUS    PORTS
fulfillment-frontend                order-fulfillment-frontend          nginx running 0.0.0.0:80->80/tcp
fulfillment-order-api               order-fulfillment-backend           running   0.0.0.0:5000->5000/tcp
fulfillment-inventory-svc           order-fulfillment-inv               running   0.0.0.0:5001->5001/tcp
fulfillment-notify-svc              order-fulfillment-notify            running   (no published port)
fulfillment-database                postgres:16                          running   0.0.0.0:5432->5432/tcp
fulfillment-redis                   redis:alpine                         running   0.0.0.0:6379->6379/tcp

```

Figure 7 — Building images and starting all containers with Docker Compose

Testing Procedure and Evidence

- Health check: GET /api/health confirms the order-api is running and connected to PostgreSQL.
- Functional test: POST /api/orders simulates placing an order and verifies stock is reserved and a notification job is queued.
- Log inspection: docker compose logs order-api and docker compose logs notification-service confirm the database connection and successful job handling.
- UI verification: the React dashboard at http://localhost is checked in a browser to confirm it displays the newly placed order with the correct status.

A terminal window titled "bash - testing & verification" showing the execution of several commands. The first command is a curl GET request to http://localhost:5000/api/health, which returns a JSON object indicating the API is "ok", "db" is "connected", and "uptime" is "12s". The second command is a curl POST request to http://localhost:5000/api/orders with a JSON body containing "productId": "p1" and "qty": 2, which returns a JSON object with "orderId": "ord_8841", "status": "placed", and "reserved": true. The third command is docker compose logs order-api --tail 5, which shows three log entries: "Connected to PostgreSQL at database:5432", "Order ord_8841 created, inventory reserved", and "Notification job queued on redis". The fourth command is docker compose logs notification-service --tail 3, which shows two log entries: "Job received: order ord_8841" and "Confirmation email sent to customer". The final two lines are comments: "# Frontend checked in browser at http://localhost - dashboard shows" and "# order ord_8841 with status 'placed', confirming end-to-end flow."

```
bash - testing & verification

$ curl http://localhost:5000/api/health
{"status":"ok","db":"connected","uptime":"12s"}

$ curl -X POST http://localhost:5000/api/orders -d '{"productId":"p1","qty":2}'
{"orderId":"ord_8841","status":"placed","reserved":true}

$ docker compose logs order-api --tail 5
order-api | [info] Connected to PostgreSQL at database:5432
order-api | [info] Order ord_8841 created, inventory reserved
order-api | [info] Notification job queued on redis

$ docker compose logs notification-service --tail 3
notify-svc | [info] Job received: order ord_8841
notify-svc | [info] Confirmation email sent to customer

# Frontend checked in browser at http://localhost - dashboard shows
# order ord_8841 with status "placed", confirming end-to-end flow.
```

Figure 8 — API health check, functional test, and log verification

11. Benefits of Git and Docker

Aspect	Contribution of Git	Contribution of Docker
Collaboration	Feature branches and pull requests let multiple developers work on checkout,	A shared docker-compose.yml means every teammate runs an identical stack,

Aspect	Contribution of Git	Contribution of Docker
	inventory, and notification features in parallel without stepping on each other.	eliminating "works on my machine" issues.
Version Management	Full history, tags (v1.0.0), and diffs make it possible to trace and revert any change to the codebase.	Image tags (order-api:1.0.0) version the runtime environment alongside the code.
Portability	The repository (code + config) can be cloned on any machine with Git installed.	Containers run identically on a developer laptop, CI server, or cloud host, regardless of underlying OS.

12. Challenges and Improvements

- Merge conflicts arose when two feature branches modified the same route file (backend/src/routes/orderRoutes.js); resolved manually by reviewing both diffs before merging.
- Keeping the .env file out of Git while still documenting required variables required maintaining a separate .env.example.
- Ensuring the order-api container could reach the database and redis containers required understanding Docker's internal DNS (service name resolution) rather than using localhost.

Future Enhancements

- Add a CI/CD pipeline (e.g., GitHub Actions) to automatically run tests and build/push Docker images on every merge to main.
- Introduce a healthcheck: block in docker-compose.yml so dependent services wait until PostgreSQL is actually ready, not just started.
- Adopt semantic versioning with automated changelog generation from Conventional Commits.
- Add an NGINX reverse proxy / API gateway container in front of frontend and order-api for centralized TLS termination and routing.

Conclusion

The Smart E-commerce Order Fulfillment Platform shows Git and Docker working together effectively: Gitflow branching enabled parallel feature development and gave a real, resolved merge conflict as proof the workflow holds up in practice, while Docker and Docker Compose turned a five-service stack (frontend, order-api, notification-service) into a single-command, portable deployment. Together they meet the project's goals of collaborative version control and consistent deployment with CI/CD, health checks, and orchestration as natural next steps for production readiness.